7 — Depuração e testes

Escola Tecnológica FPFtech

Curso: Técnico em Automação Industrial

Linguagem de Programação Voltada a Microcontroladores

Semana 02

Prof. Denis Moraes Guimarães

30/09/2026

7 — Depuração e testes: percurso

Ao final da aula, você deverá conseguir

Reproduzir um defeito, localizar sua causa e verificar a correção com casos de teste definidos antes da execução.

1. 7.1 Técnicas de depuração: rastreamento, mensagens e GDB.
2. 7.2 Identificação e correção de erros.
3. 7.3 Testes básicos: casos normais, limites e entradas inválidas.

Contexto: contar leituras que atingem o limite de alarme.
Pré-requisitos: condições, laços, vetores e funções em C11.

O programa compilou. A regra está correta?

Especificação do monitor

Conte uma ocorrência para cada temperatura **maior ou igual** a 30 °C.

Leitura	28	30	32
Conta?	Não	Sim	Sim

- Resultado esperado: 2 ocorrências.
- Resultado observado: 1 ocorrência.
- A diferença indica uma falha; ainda precisamos localizar sua causa.

Um caso mínimo e reproduzível ajuda mais que trocar várias linhas ao acaso.

Depuração e teste têm papéis diferentes

Testar

Executar ou analisar casos escolhidos e comparar resultados com uma expectativa.

Pergunta: o programa atende à regra neste caso?

Depurar

Investigar uma falha, localizar sua causa e corrigir o programa.

Pergunta: por que o resultado divergiu?

Depois da correção, repita o caso que falhou e os demais casos: isso ajuda a detectar regressões.

7.2 — Em que etapa o erro aparece?

Categoria	Exemplo	Evidência
Sintaxe/tipos	Falta de ; ou tipos incompatíveis	Diagnóstico na compilação.
Ligação	Função declarada sem implementação ligada	Referência indefinida.
Execução	Acesso além do vetor	Pode falhar ou parecer funcionar; comportamento indefinido.
Lógica	> em vez de >=	Resultado incorreto em caso válido.

Atenção

Comportamento indefinido não tem saída previsível e não precisa causar uma mensagem de erro.

Primeira ferramenta: avisos do compilador

```
gcc -std=c11 -Wall -Wextra -Wpedantic \
    -g -O0 exemplos/monitor.c exemplos/regras.c \
    -o /tmp/aula30_monitor
```

- `-std=c11`: seleciona a versão da linguagem.
- `-Wall -Wextra -Wpedantic`: habilitam grupos úteis de avisos; não detectam todos os erros.
- `-g`: inclui informações para o depurador.
- `-O0`: facilita relacionar as instruções ao código-fonte.

Leia o primeiro diagnóstico, confira arquivo e linha, corrija e recompile.
Um erro pode gerar vários diagnósticos posteriores.

7.1 — Um procedimento de investigação

1. Escreva a regra e o resultado esperado.
2. Registre os dados que reproduzem a falha.
3. Reduza o cenário sem eliminar o problema.
4. Observe valores e formule uma hipótese.
5. Altere uma causa por vez.
6. Execute novamente e acrescente um teste de regressão.

Registro mínimo

Versão do código, comando, entrada, esperado, observado e alteração que resolveu a falha.

Reproduzir antes de corrigir

Campo	Registro do defeito
Entrada	Vetor `{28, 30, 32}`, limite 30.
Esperado	2, porque 30 e 32 atingem o limite.
Observado	1.
Caso reduzido	Vetor `{30}`, limite 30: esperado 1, observado 0.
Hipótese	A comparação desconsidera a igualdade.

O caso reduzido isola a fronteira da decisão e dispensa sensores, portas seriais e temporização.

Rastreamento manual do estado

```
size_t total = 0;
for (size_t i = 0; i < quantidade; ++i) {
    if (valores[i] >= limite) ++total;
}
```

i	valores[i]	Teste >= 30	total após passo
0	28	falso	0
1	30	verdadeiro	1
2	32	verdadeiro	2

Invariante: antes do passo `i`, `total` conta as ocorrências nos elementos de índices menores que `i`.

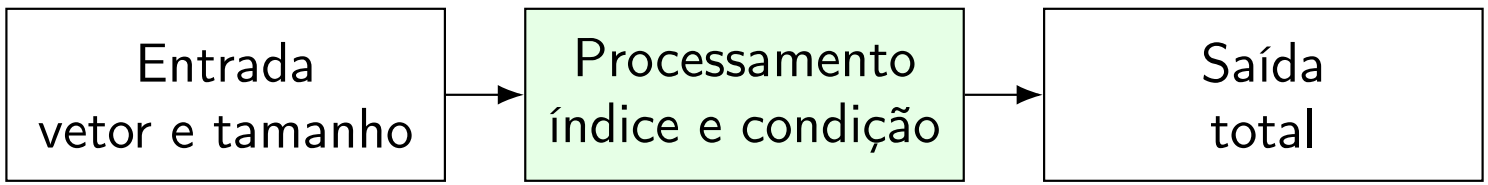
Mensagens temporárias com printf

```
for (size_t i = 0; i < quantidade; ++i) {
    printf("i=%zu valor=%d limite=%d total_antes=%zu\n",
           i, valores[i], limite, total);
    if (valores[i] >= limite) ++total;
    printf("total_depois=%zu\n", total);
}
```

- Registre valores que confirmem ou refutem uma hipótese.
- %zu corresponde a `size_t`; %d, a `int`.
- Nomeie os campos para interpretar o registro depois.
- Remova ou desative mensagens temporárias ao concluir.

Em hardware, mensagens podem mudar a temporização; a observação também tem custo.

Escolher onde observar



- Entrada errada: investigue aquisição e validação.
- Entrada certa, estado errado: investigue laço e condição.
- Retorno certo, tela errada: investigue formatação e apresentação.

Uma função pura reduz o número de lugares em que a falha pode estar.

GDB: pausar antes de adivinhar

```
gcc -std=c11 -Wall -Wextra -Wpedantic \
    -g -O0 exemplos/monitor.c exemplos/regras.c \
    -o /tmp/aula30_monitor
gdb /tmp/aula30_monitor
```

```
(gdb) break contar_no_limite
(gdb) run
(gdb) print quantidade
(gdb) print limite
(gdb) print valores[1]
```

Na primeira parada, os valores esperados são 3, 30 e 30. Um ponto de parada suspende o programa antes de continuar sua execução.

GDB: comandos para investigar uma hipótese

Comando	Uso
`list`	Exibir linhas próximas da posição atual.
`next`	Avançar uma linha sem entrar em uma chamada.
`step`	Avançar entrando em função quando aplicável.
`print total`	Mostrar o valor atual, após a inicialização.
`display i`	Mostrar automaticamente, quando estiver no escopo.
`backtrace`	Examinar a sequência de chamadas ativas.
`continue`	Retomar até a próxima parada ou o fim.
`quit`	Encerrar o depurador.

Observe a seta da próxima linha: uma atribuição nessa linha ainda pode não ter sido executada.

Erro de limite: um índice a mais

```
/* INCORRETO: fragmento para analise, nao executar. */
for (size_t i = 0; i <= quantidade; ++i) {
    if (valores[i] >= limite) ++total;
}
```

- Para 3 elementos, os índices válidos são 0, 1 e 2.
- Quando `i == quantidade`, ocorre acesso fora do vetor.
- Esse acesso tem comportamento indefinido em C.

```
/* CORRETO: intervalo [0, quantidade). */
for (size_t i = 0; i < quantidade; ++i) {
    if (valores[i] >= limite) ++total;
}
```

Erro lógico: a igualdade foi esquecida

```
/* INCORRETO para a especificacao desta aula. */
if (valores[i] > limite) ++total;

/* CORRETO: atingir o limite tambem conta. */
if (valores[i] >= limite) ++total;
```

Escolha um teste que distinga as versões

Com leitura 32 e limite 30, ambas contam. Com leitura 30 e limite 30, apenas a versão correta conta.

O compilador não conhece o significado de “atingir o limite”. Um caso de fronteira verifica essa regra.

Implementação corrigida e contrato

```
size_t contar_no_limite(const int valores[],
                        size_t quantidade, int limite)
{
    size_t total = 0;
    for (size_t i = 0; i < quantidade; ++i) {
        if (valores[i] >= limite) ++total;
    }
    return total;
}
```

- Para quantidade positiva, o vetor deve conter esse número de elementos legíveis.
- Para quantidade zero, nenhum elemento é acessado e o resultado é zero; aceita-se `NULL` nesse caso.
- A função não modifica o vetor nem realiza entrada/saída.

7.3 — Definir casos antes de executar

Caso	Leituras	Limite	Esperado
Normal misto	`{28, 30, 32}`	30	2
Nenhuma ocorrência	`{25, 28}`	30	0
Todas as ocorrências	`{30, 31}`	30	2
Igualdade	`{30}`	30	1
Vazio	Nenhuma leitura	30	0
Valores negativos	`{-2, 0, 2}`	0	2

O esperado deve vir da especificação, não da execução do próprio código que será testado.

Fronteiras e partições úteis

Próximo do limite 30

Teste 29, 30 e 31: imediatamente abaixo, exatamente igual e imediatamente acima.

- Quantidade: zero, um e vários elementos.
- Resultado: nenhuma, algumas e todas as leituras contadas.
- Dados: negativos, zero e positivos, quando permitidos.
- Repetições: leituras iguais em posições diferentes.

Muitos testes aleatórios podem deixar uma fronteira importante sem cobertura. Escolha casos por comportamento esperado.

Entrada inválida: validar na fronteira do sistema

```
bool temperatura_valida(int temperatura_c)
{
    return temperatura_c >= -40 && temperatura_c <= 125;
}
```

Faixa **didática** adotada neste exemplo; não representa todo sensor.

- -40 e 125 são aceitos; -41 e 126 são rejeitados.
- Leituras rejeitadas não seguem para a decisão de alarme.
- Não teste a função de contagem com um ponteiro inválido ou tamanho inventado: isso viola seu contrato.

Texto não numérico deve ser rejeitado na conversão de entrada, antes desta função que já recebe um inteiro.

Assert: uma expectativa executável

```c
#include <assert.h>

const int leituras[] = {28, 30, 32};
assert(contar_no_limite(leituras, 3, 30) == 2);
```

- Se a expressão for falsa, a execução de teste é interrompida com diagnóstico.
- Se todas forem verdadeiras, o programa continua.
- Compilar com `NDEBUG` definido desativa `assert`.
- Não use `assert` como única validação de entradas externas.

Evite efeitos necessários dentro de `assert`; eles desapareceriam quando as verificações fossem desativadas.

Um conjunto automatizado de casos

```
const int misto[] = {28, 30, 32};
const int nenhum[] = {25, 28};
const int todos[] = {30, 31};
const int igual[] = {30};
const int negativos[] = {-2, 0, 2};
assert(contar_no_limite(misto, 3, 30) == 2);
assert(contar_no_limite(nenhum, 2, 30) == 0);
assert(contar_no_limite(todos, 2, 30) == 2);
assert(contar_no_limite(igual, 1, 30) == 1);
assert(contar_no_limite(NULL, 0, 30) == 0);
assert(contar_no_limite(negativos, 3, 0) == 2);
puts("6 testes de contagem passaram");
```

Versão completa: `exemplos/testes.c`. Ela também verifica seis casos da validação de temperatura.

Executar os testes e interpretar o resultado

```
gcc -std=c11 -Wall -Wextra -Wpedantic \
    exemplos/testes.c exemplos/regras.c \
    -o /tmp/aula30_testes
/tmp/aula30_testes
```

```
6 testes de contagem passaram
6 testes de validacao passaram
```

- As mensagens aparecem após as verificações bem-sucedidas.
- Uma falha interrompe a execução antes da mensagem correspondente.
- Passar nos casos escolhidos aumenta a confiança; não prova ausência de todos os defeitos.

Teste de regressão: guardar o caso que revelou a falha

1. Preserve o caso {30}, limite 30, esperado 1.
2. Confirme que a comparação antiga > falha nesse caso.
3. Corrija para >= e execute o conjunto completo.
4. Mantenha o caso nas próximas versões.

Duas fronteiras diferentes

`i < quantidade` protege o acesso ao vetor.
`valores[i] >= limite` implementa a decisão de alarme.

Um teste de saída não garante sozinho que todos os acessos à memória foram válidos; revise também os limites do laço.

Verificação adicional de memória no computador

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g -O0 \
    -fsanitize=address,undefined \
    exemplos/testes.c exemplos/regras.c \
    -o /tmp/aula30_sanitizado
/tmp/aula30_sanitizado
```

- Ferramentas opcionais detectam classes de acesso inválido e comportamento indefinido durante a execução.
- Exigem suporte do compilador e do ambiente.
- Complementam avisos, revisão de código e testes de resultado.
- Sua disponibilidade e custo diferem no microcontrolador.

O exemplo fornecido está corrigido. Os fragmentos defeituosos ficam em um arquivo de texto para análise.

Prática guiada — Investigar com evidências

1. Compile `monitor.c` com avisos e símbolos de depuração.
2. Confira o resultado: 2 ocorrências em três leituras.
3. No GDB, pare em `contar_no_limite`; examine quantidade, limite e `valores[1]`.
4. Avance pelo laço e registre quando o total muda.
5. Rode `testes.c` e identifique o caso que verifica a igualdade.

Entrega

Tabela com índice, leitura e total após cada passo; comando de compilação; evidência de testes concluídos.

Exercícios — Prever, classificar e corrigir

1. Classifique: falta de `;`, função ausente na ligação, acesso `vetor[n]` e uso de `>` para limite inclusivo.
2. Para `{29,30,31,30}` e limite 30, determine o total e escreva um `assert`.
3. Por que o teste com `{32}` não detecta o erro de comparação? Proponha um que detecte.
4. Teste a validação nos quatro valores próximos às extremidades: -41, -40, 125 e 126.

Desafio: acrescente o vetor `{29,30,31,30}` à suíte e atualize a mensagem de quantidade de casos.

Conferência dos exercícios

- Categorias: sintaxe; ligação; execução com comportamento indefinido; lógica.
- O vetor tem três ocorrências: 30, 31 e 30.

```
const int fronteiras[] = {29, 30, 31, 30};
assert(contar_no_limite(fronteiras, 4, 30) == 3);
```

- 32 satisfaz > e >=; use 30 para distinguir.
- Validação: −41 rejeitado; −40 aceito; 125 aceito; 126 rejeitado.
- Na prática, os totais após cada passo são 0, 1 e 2.

Síntese — Corrigir e demonstrar a correção

- Uma falha precisa de entrada, esperado e observado.
- Avisos, rastreamento e GDB ajudam a localizar a causa.
- Limite do vetor e limite da decisão são condições distintas.
- Casos normais, fronteiras e validação exercitam comportamentos diferentes.
- Um teste de regressão preserva a evidência da correção.

Verificação de saída

Você consegue mostrar um caso que falhava antes da correção e passa depois dela?

Próxima aula: aplicar decisões testadas a entradas e saídas digitais, observando também as limitações do hardware.